Course 2: Tokenization

What is tokenization?

Turning text...

```
I love playing soccer!
```

...into *tokens*

```
['I', 'love', 'play', 'ing', 'soccer', '!']
```

Historical Notions

Tokenization Origins

The word token comes from linguistics

“ *non-empty contiguous sequence of graphemes or phonemes in a document* ”

≈

Split text on blanks

Tokenization Origins

```
old_tokenize("I love playing soccer!") = ['I', 'love', 'playing', 'soccer!']
```

- Different from *word-forms* ⚠️
  - *damélo* → *da/ mé/ lo* (=*give/ me/ it*)

Tokenization Origins

Natural language is split into...

- Sentences, utterances, documents... (*macroscopical*) that are split into...
  - Tokens, word-forms... (*microscopical*)

→ Used for linguistic tasks (POS tagging, syntax parsing,...)

Tokenization & ML

ML-based NLP (usually) relies on **sub-word** tokenization:

- Gives better performance
- **Fixed-size vocabulary** often required
  - Out-Of-Vocabulary (OOV) issue

Tokenization & ML

Evolution of modeling complexity w.r.t. the sequence length n

Model Type	Year	Complexity
Tf-Idf	1972	O(1)
RNNs	~1985	O(n)
Transformers	2017	$O(n^2)$

→ Long sequences (e.g. character-level) are prohibitive

Modern framework

- **Pre-tokenization**

  `"I'm lovin' it!" -> ["i", "am", "loving", "it", "!"]`

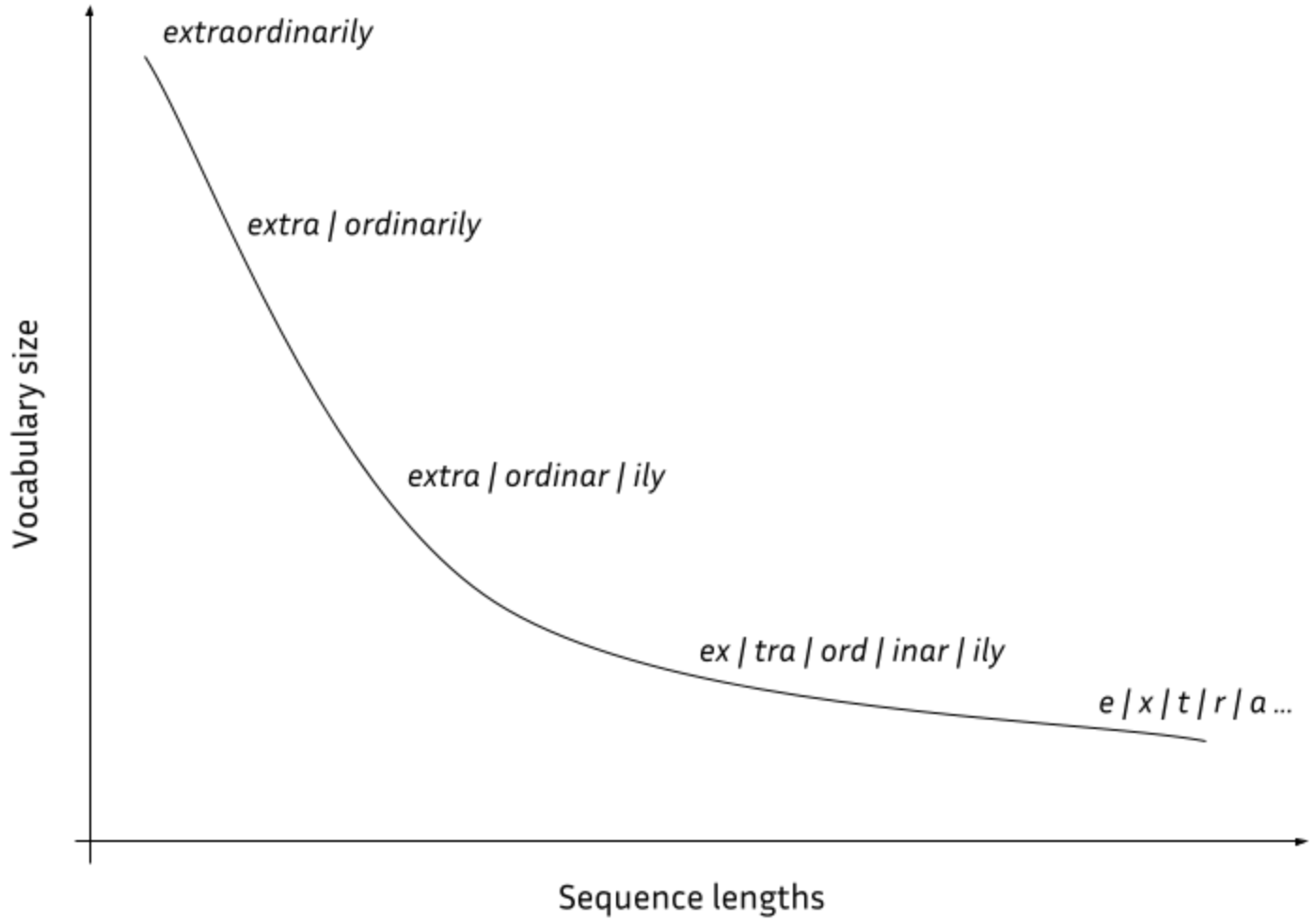
  - Normalization
    - Rules around punctuation (`_:_`, `_!`, ...)
    - Spelling correction (`"imo" -> "in my opinion"`)
    - Named entities (`"covid" -> "COVID-19"`)
    - ...
  - Rule-based segmentation
    - Blanks, punctuation, ...

Modern framework

- **Tokenization** `-> ["i", "am", "lov", "##ing", "it", "!"]`
  - Split units at subword level
  - Fixed vocabulary
  - **Trained** on text samples
  - Used in inference mode at *pre-processing* time

Sub-word Tokenization

Granularity



Granularity

→ Trade-off between short sequences and reasonable vocabulary size

Fertility

For a string sequence S:

$$\text{fertility}(S) = \frac{\#\text{ tokens}}{\#\text{ words}}$$

Algorithms

Byte-Pair Encoding (BPE)

Let's encode "*aaabdaaabac*" in an optimized way:

- Observed pairs: {*aa*, *ab*, *bd*, *da*, *ba*, *ac*}
- Observed **occurences**: {***aa*: 4**, *ab*: 2, *bd*: 1, *da*: 1, *ba*: 1, *ac*: 1}
- Set *X = aa*
- Encode *aaabdaaabac* → *XabdXabac*
- Start again from *XabdXabac*

Byte-Pair Encoding (BPE)

(current rules: *aa* → *X*)

Let's encode "*XabdXabac*" in an optimized way:

- Observed pairs: {*Xa*, *ab*, *bd*, *dX*, *ba*, *ac*}
- Observed **occurences**: {***Xa*: 2**, ***ab*: 2**, *bd*: 1, *dX*: 1, *ba*: 1, *ac*: 1}
- Set *Y = ab*
- Encode *XabdXabac* → *XYdXYac*
- Start again from *XYdXYac*

Byte-Pair Encoding (BPE)

(current rules: *aa* → *X*, *ab* → *Y*)
Let's encode "*XYdXYac*" in an optimized way:

- Observed pairs: {*XY*, *Yd*, *dX*, *Ya*, *ac*}
- Observed occurences: {***XY*: 2**, *Yd*: 1, *dX*: 1, *Ya*: 1, *ac*: 1}
- Set *Z* = *XY*
- Encode *XYdXYac* → *ZdZac*
- Start again from *ZdZac*

Byte-Pair Encoding (BPE)

(current rules: *aa* → *X*, *ab* → *Y*, *XY* → *Z*)
Let's encode "*ZdZac*" in an optimized way:

- Observed pairs: {*Zd*, *dZ*, *Za*, *ac*}
- Observed occurences: {*Zd*: 1, *dZ*: 1, *Za*: 1, *ac*: 1}
- **All pairs are unique => END**

Byte-Pair Encoding (BPE)

Final encoding: *aaabdaaabac* → *ZdZac*

with **merge rules**:

1. *aa* → *X*
2. *ab* → *Y*
3. *XY* → *Z*

Decoding: follow merge rules in opposite order

BPE Training - pre-tokenization

```
training_sentences = [
    "Education is very important!",
    "A cat and a dog live on an island",
    "We'll be landing in Cabo Verde",
]
```

=>

```
pretokenized = ["education_", "is_", "very_", "important_", "!_", "a_",
         "cat_", "and_", "a_", "dog_", "live_", "on_", "an_", "island_",
         "we", "'", "ll_", "be_", "landing_", "in_", "cabo_" "Verde_"
]
```

BPE Training - iteration 1

```
tokenized = [
    ['e', 'd', 'u', 'c', 'a', 't', 'i', 'o', 'n', '_'], ..., ['i', 'm', 'p', 'o', 'r', 't', 'a', 'n', 't', '_'], ['!', '_'],
    ['a', '_'], ['c', 'a', 't', '_'], ['a', 'n', 'd', '_'],  ..., ['o', 'n', '_'], ['a', 'n', '_'], ['i', 's', 'l', 'a', 'n', 'd', '_'],
    ['w', 'e'], ["'"], ['l', 'l', '_'], ['b', 'e', '_'], ['l', 'a', 'n', 'd', 'i', 'n', 'g', '_'], ..., ['v', 'e', 'r', 'd', 'e', '_']
]
```

→ Most common pair: "an"

```
tokenized = [
    ['e', 'd', 'u', 'c', 'a', 't', 'i', 'o', 'n', '_'], ..., ['i', 'm', 'p', 'o', 'r', 't', 'an', 't', '_'], ['!', '_'],
    ['a', '_'], ['c', 'a', 't', '_'], ['an', 'd', '_'],  ..., ['o', 'n', '_'], ['an', '_'], ['i', 's', 'l', 'an', 'd', '_'],
    ['w', 'e'], ["'"], ['l', 'l', '_'], ['b', 'e', '_'], ['l', 'an', 'd', 'i', 'n', 'g', '_'], ..., ['v', 'e', 'r', 'd', 'e', '_']
]
```

BPE Training - iteration 2

```
tokenized = [
    ['e', 'd', 'u', 'c', 'a', 't', 'i', 'o', 'n', '_'], ..., ['i', 'm', 'p', 'o', 'r', 't', 'an', 't', '_'], ['!', '_'],
    ['a', '_'], ['c', 'a', 't', '_'], ['an', 'd', '_'],  ..., ['o', 'n', '_'], ['an', '_'], ['i', 's', 'l', 'an', 'd', '_'],
    ['w', 'e'], ["'"], ['l', 'l', '_'], ['b', 'e', '_'], ['l', 'an', 'd', 'i', 'n', 'g', '_'], ..., ['v', 'e', 'r', 'd', 'e', '_']
]
```

→ Most common pair: `"ca"`

```
tokenized = [
    ['e', 'd', 'u', 'ca', 't', 'i', 'o', 'n', '_'], ..., ['i', 'm', 'p', 'o', 'r', 't', 'an', 't', '_'], ['!', '_'],
    ['a', '_'], ['ca', 't', '_'], ['an', 'd', '_'],  ..., ['o', 'n', '_'], ['an', '_'], ['i', 's', 'l', 'an', 'd', '_'],
    ['w', 'e'], ["'"], ['l', 'l', '_'], ['b', 'e', '_'], ['l', 'an', 'd', 'i', 'n', 'g', '_'], ..., ['v', 'e', 'r', 'd', 'e', '_']
]
```

BPE Training - iteration 14 (final)

```
tokenized = [
    ['e', 'd', 'u', 'cat', 'i', 'on_'], ['is', '_'], ['ver', 'y', '_'], ['i', 'm', 'p', 'o', 'r', 't', 'an', 't', '_'], ['!', '_'],
    ['a_'], ['cat', '_'], ['and_'], ['a_'], ..., ['on_'], ['an', '_'], ['is', 'l', 'and_'],
    ['w', 'e'], ["'"], ..., ['l', 'and', 'i', 'n', 'g_'], ['i', 'n_'], ['ca', 'b', 'o', '_'], ['ver', 'd', 'e_']
]
```

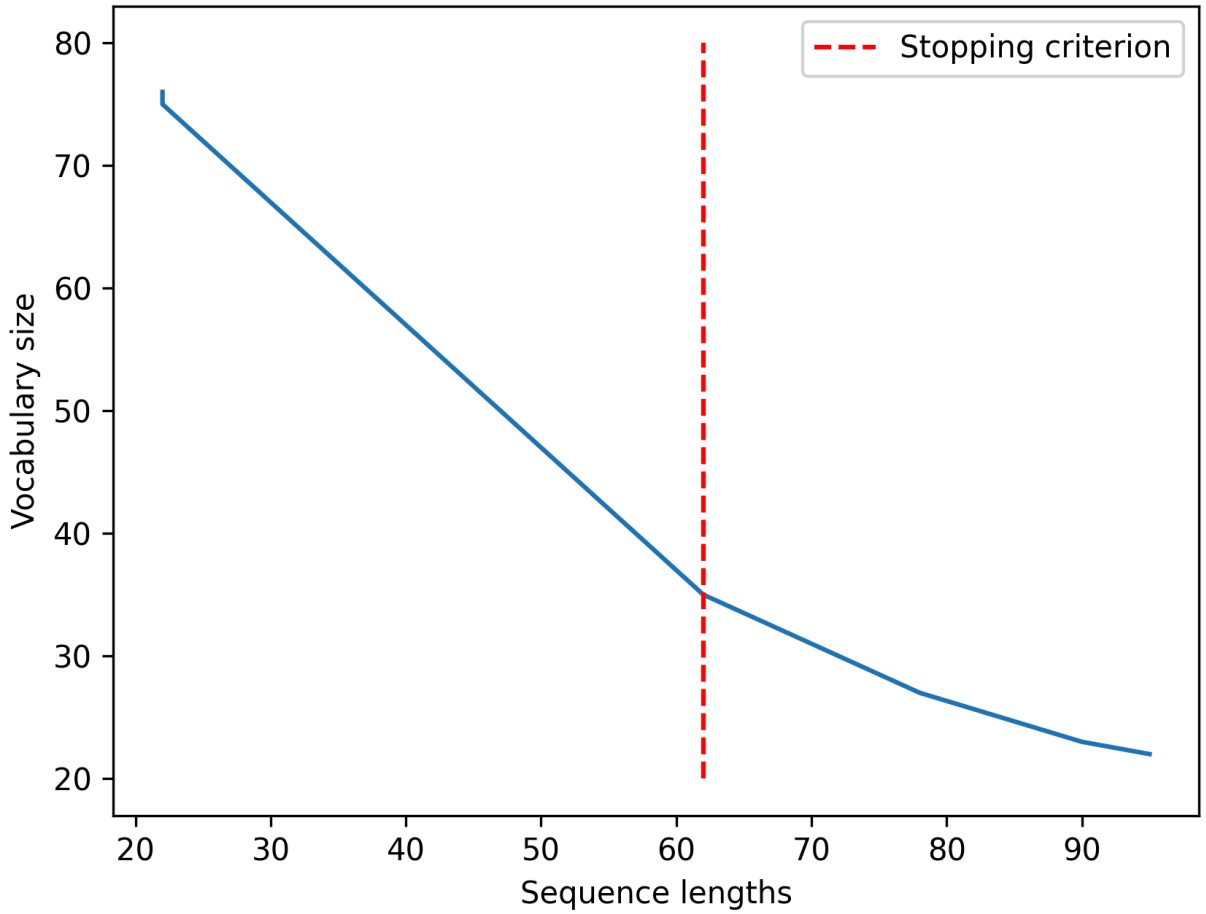
"Created" tokens:

```
['an', 'ca', 'n_', 've', 'and', 'cat', 'on_', 'is', 'ver', 'a_', 'and_', 'g_', 'e_']
```

→ English common words (a, and, on, is, ...)

→ `and` vs `and_`

BPE - Granularity



WordPiece

- Based on merge rules too
- Initial processing is different:

BPE:

```
["education", "is"] => [["e", "d", "u", ..., "n", "_"], ["i", "s", "_"]]
```

WordPiece:

```
["education", "is"] => [["e", "##d", "##u", "##c",...], ["i", "##s"]]
```

WordPiece

- Pairs are scored using this score function:

$$S((t_1, t_2)) = \frac{freq(t_1 t_2)}{freq(t_1) freq(t2)}$$

- if t_1 and t_2 are common, less likely to merge
  - ex: *dream/##ing* → not merged
- if t_1 and t_2 are rare but $t_1 t_2$ is common, more likely to merge
  - ex: *pulv/##erise* → *pulverise*

The Unigram Philosophy

Don't commit to one segmentation: model the uncertainty!

Core Idea: Treat segmentation as a probability distribution

- Each token has a **learned probability**
- Each segmentation has probability = product of its token probabilities
- Text probability = sum over all possible segmentations

Historical Context

$$P(x_1, x_2, \ldots, x_n) = P(x_1) \times P(x_2) \times \ldots \times P(x_n)$$

$$P(\text{text}) = \sum_{\text{segmentations}} P(\text{segmentation})$$

Key Advantage: Unigram explicitly models ambiguity during training

=> More robust to novel words and rare combinations

Training Unigram: High-Level Overview

Step 1: Initialize large vocabulary (all substrings up to a given length)

Step 2: Set uniform probabilities: $P(\text{token}) = 1/|V|$

Step 3: Expectation-maximization (EM) algorithm (iterate until convergence)

- **E-step:** Sample segmentations, count tokens
- **M-step:** Update $P(\text{token}) \propto \text{count}(\text{token})$

Step 4: Prune low-probability tokens

Step 5: Repeat steps 3-4 until vocabulary reaches target size

Training Example: Setup

Training Corpus (3 documents):

```
Doc 1: "hello␣world"
Doc 2: "hello␣there"
Doc 3: "goodbye␣world"
```

Initial Vocabulary (25 tokens):

- **Characters:** h, e, l, o, w, r, d, t, g, b, y, ␣
- **Bigrams:** he, el, ll, lo, or, ld, th, er, re
- **Words:** hello, world, there, goodbye

Initialize: $P(\text{token}) = 1/25 = 0.04$ for all tokens

Forward Filtering: Computing Probability Mass

Goal: Compute $P(\text{text})$ = sum over all possible segmentations

Problem: Exponentially many segmentations: $\mathcal{O}(2^n)$ for a string of length n.

Solution: Dynamic Programming

Key Idea: Compute $\alpha[t]$ = total probability of reaching position t

$$\alpha[0] = 1.0 \quad (\text{base case})$$

$$\alpha[t] = \sum_{s<t} \alpha[s] \times P(\text{token from } s \text{ to } t)$$

At the end: $\alpha[n] = P(\text{text})$, the total probability mass.

Forward Filtering: "hello␣world" (abbreviated)

```
Position 0: α[0] = 1.0
```

```
Position 1 (after "h"):
  - Token "h": α[0] × P("h") = 1.0 × 0.04 = 0.04
  -> α[1] = 0.04
```

```
Position 2 (after "he"):
  - Token "e": α[1] × 0.04 = 0.0016
  - Token "he": α[0] × 0.04 = 0.04
  -> α[2] = 0.0016 + 0.04 = 0.0416
```

```
Position 5 (after "hello"):
  - Token "hello": α[0] × 0.04 = 0.04
  - Other paths: ~0.0002
  -> α[5] ≈ 0.0402
```

Key insight: "hello" as single token dominates the probability mass!

Backward Sampling: Drawing Segmentations

Goal: Sample segmentations proportionally to their probability

Algorithm:

1. Start at the end of the text
2. For each token that could end here, compute conditional probability:

$$P(\text{token}|\text{at position } t) = \frac{\alpha[s] \times P(\text{token})}{\alpha[t]}$$

3. Sample randomly according to these probabilities
4. Move backward to the start of the chosen token
5. Repeat until reaching position 0

This whole loop is called **forward filtering backward sampling (FFBS)**.

Sampling "hello␣world": Three Samples

Sample 1: `["hello", "␣", "world"]`

Sample 2: `["he", "ll", "o", "␣", "w", "or", "ld"]`

Sample 3: `["hello", "␣", "w", "or", "ld"]`

We sample multiple times per document to get diverse training signal.

Then we count how often each token appeared across all samples.

E-Step: Accumulating Token Counts

After sampling all documents multiple times, we count token occurrences:

Token	Count
"hello"	8 times
"goodbye"	4 times
"world"	3 times
"there"	2 times
"␣"	18 times
"l"	7 times
"o"	6 times
... others ...	52 times
Total	**100 times**

M-Step: Updating Probabilities

Maximum Likelihood Estimation: $P(\text{token}) \propto \text{count}(\text{token})$

Token	Count	Old P	New P
"␣"	18	0.04	**0.18** ↑
"hello"	8	0.04	**0.08** ↑
"goodbye"	4	0.04	0.04
"world"	3	0.04	0.03
"h"	1	0.04	**0.01** ↓

EM Iterations: Convergence

After iteration 1, probabilities shift toward useful tokens.
In iteration 2, these tokens get sampled more frequently.

After 10 iterations (convergence):

Token	Probability	Note
"␣"	0.25	↑ Essential separator
"hello"	0.15	↑ Common word
"goodbye"	0.12	↑ Common word
"world"	0.10	↑ Common word
"ll"	0.06	Useful bigram
"h"	0.01	↓ Rarely needed

Vocabulary Pruning

Problem: Initial vocabulary is huge (100K+ tokens)
Goal: Reduce to manageable size (8K-32K tokens)

- Start from a very big vocabulary V.
- Infer on all pre-tokenized units $w \in W$ and get total score as:

$$score(V, W) = \sum_{w=(t_1 \ldots t_n) \in W} -\log(P_V(t_1) \times \ldots \times P_V(t_n))$$

- For all token t, compute $score(V - \{t\}, W)$
- Get rid of the 20% tokens that **least decrease** the score when removed
- Iterate 🔁 (until you have desired vocabulary size)

⚠️ **Always keep character-level tokens for coverage.**

Training vs. Inference: Different Algorithms

Training (Offline)	Inference (Online)
Forward-Backward + Sampling	Viterbi Algorithm
Sample **multiple** segmentations	Finds **single best** segmentation
Explores diverse strategies	Uses max instead of sum
Stochastic	**Deterministic**
Builds robust vocabulary	Fast and predictable

Viterbi Decoding: Finding the Best Path

Goal: Find the highest-probability segmentation (not sum over all)

Algorithm: Same as forward pass, but use **MAX** instead of **SUM**

$$\beta[0] = 1.0 \quad (\text{base case})$$

$$\beta[t] = \max_{s<t}\{\beta[s] \times P(\text{token from } s \text{ to } t)\}$$

- Track which token gave the max at each position (**backpointer**)
- At the end, follow backpointers to reconstruct the best path

Viterbi Training: Step-by-Step for "email"

Token	Freq. in Corpus	Probability (Freq / 42)
`e`	6	0.143
`m`	4	0.095
`a`	3	0.071
`i`	5	0.119
`l`	5	0.119
`em`	1	0.024
`ma`	3	0.071
`ai`	3	0.071
`il`	3	0.071
`mail`	3	0.071
`email`	1	0.024

Viterbi Training: Step-by-Step for "email"

We'll build a table to track the best score and segmentation that ends at each character.

Position 1: `e`

- **Path**: `['e']`
- **Score**: $P(e) = 0.143$
- **Best Score ending at** `e`: **0.143**

Viterbi Training: Step-by-Step for "email"

Position 2: `m` (String is "em")
We check all possible ways the segmentation can end at `m`:

1. **Ending with token** `m`: The path is `['e', 'm']`.
   - Score = (Best score for `e`) $\times P(m)$
   - Score = $0.143 \times 0.095 = 0.014$
2. **Ending with token** `em`: The path is `['em']`.
   - Score = $P(em) = 0.024$

Decision: $0.024 > 0.014$.

- **Best Score ending at** `m`: **0.024**
- **Best Path**: `['em']`

Viterbi Training: Step-by-Step for "email"

Position 3: `a` (String is "ema")

1. **Ending with** `a`: Path `['em', 'a']`
   - Score = (Best score for `em`) $\times P(a)$
   - Score = $0.024 \times 0.071 = 0.0017$
2. **Ending with** `ma`: Path `['e', 'ma']`
   - Score = (Best score for `e`) $\times P(ma)$
   - Score = $0.143 \times 0.071 = 0.01$

Decision: $0.01 > 0.0017$.

- **Best Score ending at** `a`: **0.01**
- **Best Path**: `['e', 'ma']`

Viterbi Training: Step-by-Step for "email"

Position 4: `i` (String is "emai")

1. **Ending with** `i`: Path `['e', 'ma', 'i']`
   - Score = (Best score for `ema`) $\times P(i)$
   - Score = $0.01 \times 0.119 = 0.00119$
2. **Ending with** `ai`: Path `['em', 'ai']`
   - Score = (Best score for `em`) $\times P(ai)$
   - Score = $0.024 \times 0.071 = 0.0017$

Decision: $0.0017 > 0.00119$.

- **Best Score ending at** `i`: **0.0017**
- **Best Path**: `['em', 'ai']`

Viterbi Training: Step-by-Step for "email"

Position 5: `l` (String is "email") - Final Step!

1. **Ending with** `l`: Path `['em', 'ai', 'l']`
   - Score = (Best for `emai`) $\times P(l) = 0.0017 \times 0.119 = 0.0002$
2. **Ending with** `il`: Path `['e', 'ma', 'il']`
   - Score = (Best for `ema`) $\times P(il) = 0.01 \times 0.071 = 0.00071$
3. **Ending with** `mail`: Path `['e', 'mail']`
   - Score = (Best for `e`) $\times P(mail) = 0.143 \times 0.071 = 0.01$
4. **Ending with** `email`: Path `['email']`
   - Score = $P(email) = 0.024$

Final Decision: The highest score is **0.024**.

- **Optimal Segmentation**: `['email']`

Implementation Tricks in Practice

1. Log Space Computation

- Work with $\log P$ instead of P to avoid numerical underflow
- Multiply -> Sum -> LogSumExp trick

2. Forward-Backward Algorithm (SentencePiece)

- Compute exact expected counts without sampling
- Forward pass: $\alpha[t]$ (prefix probabilities)
- Backward pass: $\beta[t]$ (suffix probabilities)

The Forward-Backward Trick

Instead of sampling, compute **exact expected counts**:

For each token at each position $[s : t]$:

$$\text{Expected count} = \frac{\alpha[s] \times P(\text{token}) \times \beta[t]}{P(\text{text})}$$

Where:

- $\alpha[s]$ = probability of all ways to reach position s
- $\beta[t]$ = probability of all ways to segment from t to end
- $P(\text{text}) = \alpha[n]$ = total probability

Limits & Alternatives

Limits

- Fixed vocabulary...
  - ... leads to OOV (out-of-vocabulary)
  - ... scales poorly to 100+ languages (and scripts)
  - ... can cause over-segmentation

```
bpe("artificial intelligence is real") => 'artificial', 'intelligence', 'is', 'real'
```

```
bpe("aritificial inteligense is reaal") =>
'ari', '##ti', '##fi', '##cial', 'intel', '##igen', '##se', 'is', 're', '##aa', '##l'
```

Alternatives - BPE dropout (Provilkov et al.)

→ Randomly removes part of the vocabulary during training

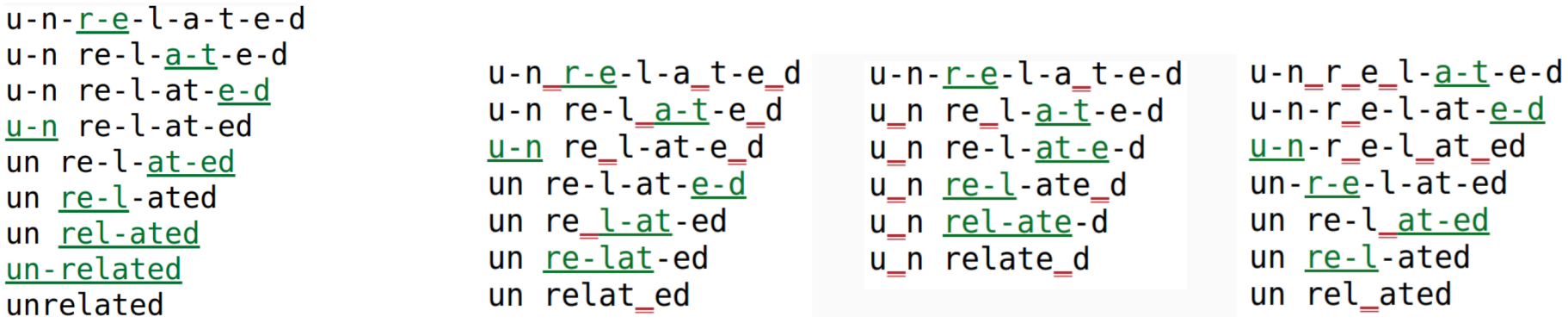
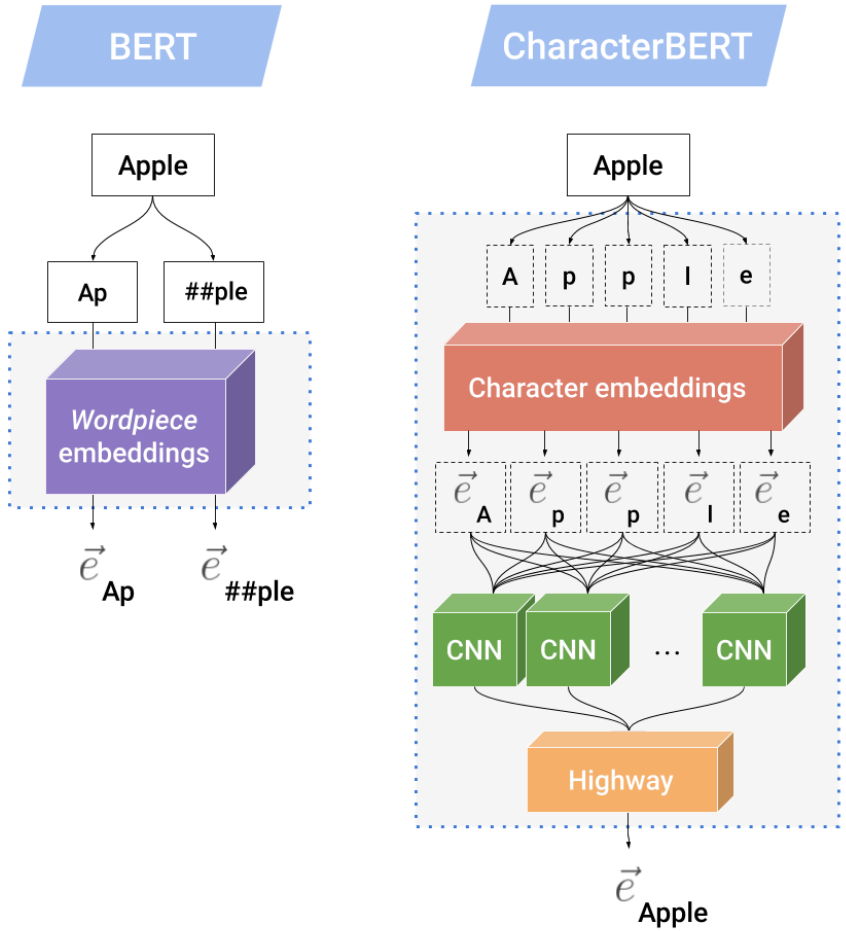


Figure 1: Segmentation process of the word '*unrelated*' using (a) BPE, (b) *BPE-dropout*. Hyphens indicate possible merges (merges which are present in the merge table); merges performed at each iteration are shown in green, dropped – in red.

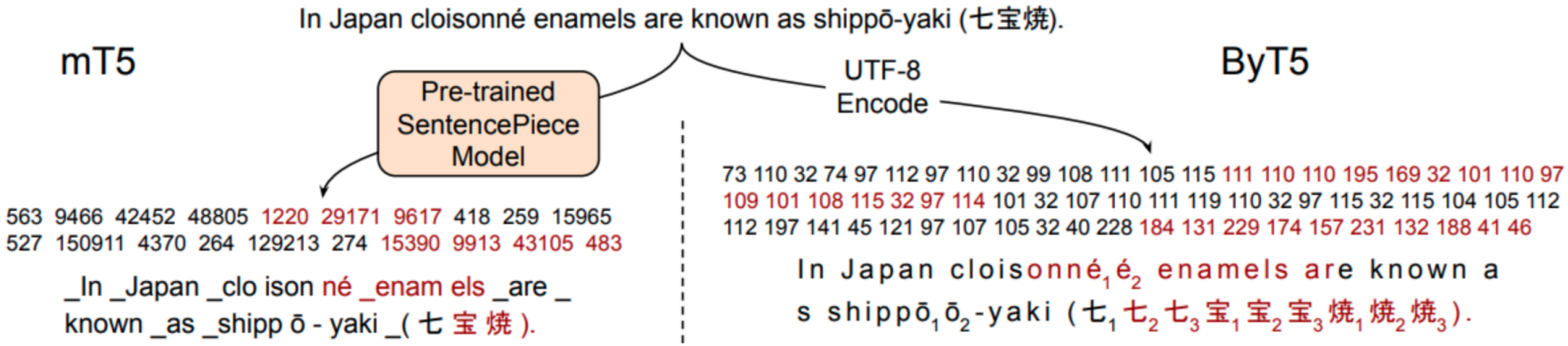
=> makes models more robust to misspellings

Alternatives - CharacterBERT (El Boukkouri et al.)



Alternatives - ByT5 (Xue et al.)

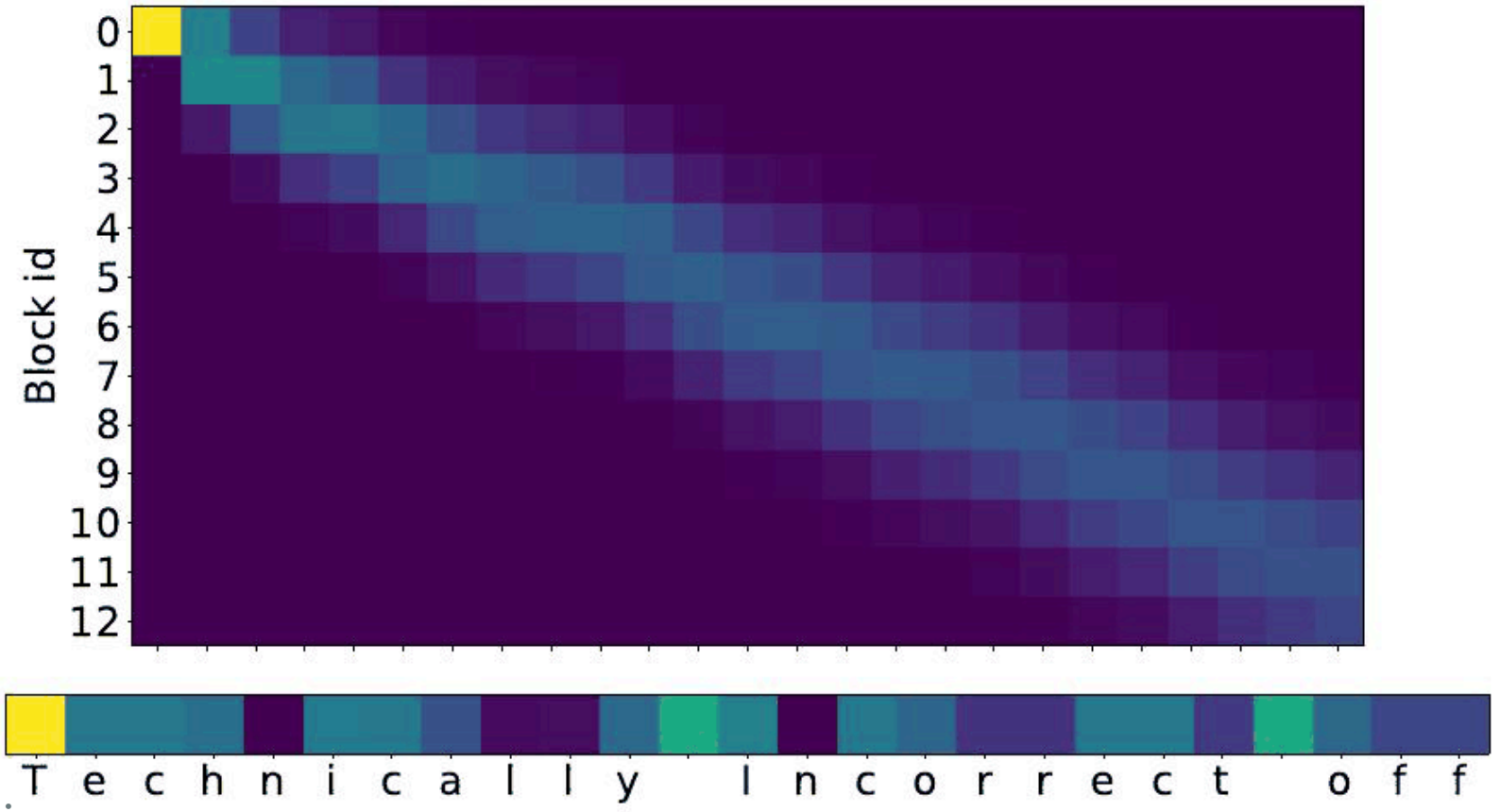
- Gives directly bytes (~characters) as inputs to the model



=> more robust and data efficient BUT ~10 times slower and more hardware consumption

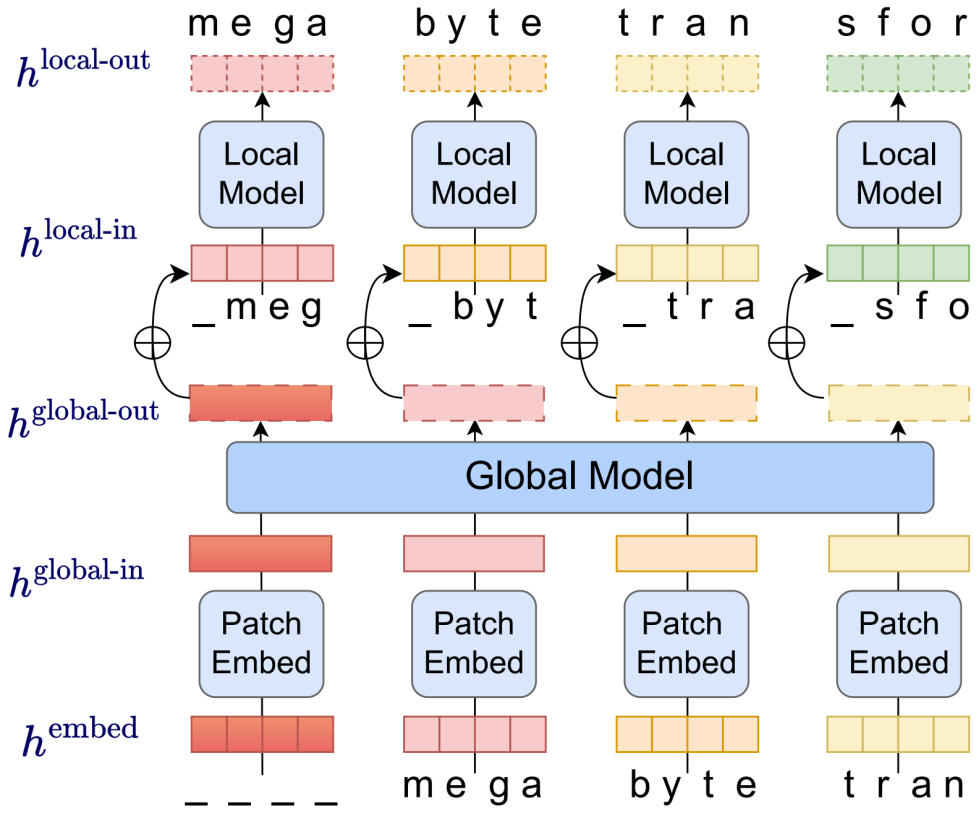
Neural tokenization - MANTa (Godey et al.)

- Allows the language model to learn its *own* mapping



Neural tokenization - MEGABYTE (Yu et al.)

- Encode and then decode autoregressively



Takeaways

- Tokenization: Art of splitting sentences/words into meaningful smaller units
- In ML: subword tokenization is (very) commonly used
- Three main algorithms
  - **BPE**: iteratively learn most frequent merges
  - **WordPiece**: BPE with adjusted frequency score
  - **Unigram**: Start big and remove tokens that won't be missed
- When facing noisy and/or complex text, alternatives exist